

Directory In linux

คืออันเดียวกันกับ **folder** แต่ทางวิศวกรรมเราเรียกว่า **directory** ซึ่งของ **Linux** นั้นจะเป็น **Directory** ที่มีโครงสร้างแตกต่างจาก **windows** อย่างสิ้นเชิง ปกติ **windows** เรามักจะเห็นเป็น **Drive** สิ่งที่เหมาะสมคือทั้ง 2 ระบบปฏิบัติการมี **directory** หรือ **folder** สำหรับ **user/program/system** เหมือนกันเพียงแค่การวางโครงสร้างนั้นแต่กันโดยมีโครงสร้างเป็นแบบต้นไม้เหมือนกัน **hierarchy** แต่ **windows** จะแบ่งออกเป็นหลาย **Drives** เหมือนมีต้นไม้หลายต้น กลับกัน **Linux** มีต้นไม้ต้นเดียว การเข้าถึงก็ต่างกัน **windows** ใช้ **backslash** ส่วน **linux** ใช้ **/** เลย รวมถึง **Linux** จะชอบแยก **Logical disk** **/home** กับ **/boot** กับ **/** กับ **swap** ออกจากกัน จะไม่ได้แยกเป็น **drive** รวมถึง **file system** จะต่างกันเช่น **windows** อาจจะใช้ **NTFS** หรือ **FAT32** แต่ **Linux** ใช้ **Ext4/Btrfs/XFS/swap partition** แต่จริงๆ **windows** ก็มี **memory page** หน้าที่หลักเหมือนกันกับ **swap partition** เลยเพียงแต่ **linux** แบบออกเป็น **partition** แยกไปเลยก็ได้หรือไม่แบ่งออกเป็น **partition** แยกออกไปเลยก็ได้เหมือนกัน

Basic command

Pwd

Cd เช่น **cd ..** หรือ . #จุดเดียวคือการบอกที่อยู่ปัจจุบัน .. คือหมายถึงที่อยู่ก่อนหน้า

Ls/lc -al

Finger

Whoami/who

Logout/exit

Mkdir/rmdir

python3

Gcc -o [path] [path]

นอกจากคำสั่งพื้นฐานแล้วขอแนะนำให้อู้จัก **Editor** ชื่อว่า **Vi** หรือเราจะใช้ว่า **Vim** ก็ได้เหมือนกันซึ่งเป็น **Editor** ที่ทรงพลังและเบาเครื่องๆ มากๆ นิยมใช้กับเครื่องที่เป็น **kernel Linux** หรือ **Unix kernel** นั่นคือเบรียมเหมือนลานแปรรภาษาที่อยู่ใกล้ชิดกับเครื่องมากที่สุด

คู่มือการใช้งาน Vi หรือ Vim

I = insert mode

a = also insert mode but write after cursor

dd = delete hold line

yy = copy hold line

p = paste

x = delete under cursor

Esc = Back to normal mode

o = insert line below

O = insert line above

Colon(:) = q(exit if no save had changed)/q!(no save)/qw(save)

Example :q!(quit with no changed) :qw(save and quit) :q(quite if no changed)

ASCII (American Standard Codeb for International Interchange)

ASCII	Char	Hex	Bin	ASCII	Char	Hex	Bin	ASCII	Char	Hex	Bin	ASCII	Char	Hex	Bin
65	A	41	0100 0001	97	a	61	0110 0001	32	space	20	0010 0000	54	6	36	0011 0110
66	B	42	0100 0010	98	b	62	0110 0010	33	!	21	0010 0001	55	7	37	0011 0111
67	C	43	0100 0011	99	c	63	0110 0011	34	"	22	0010 0010	56	8	38	0011 1000
68	D	44	0100 0100	100	d	64	0110 0100	35	#	23	0010 0011	57	9	39	0011 1001
69	E	45	0100 0101	101	e	65	0110 0101	36	\$	24	0010 0100	58	:	3A	0011 1010
70	F	46	0100 0110	102	f	66	0110 0110	37	%	25	0010 0101	59	;	3B	0011 1011
71	G	47	0100 0111	103	g	67	0110 0111	38	&	26	0010 0110	60	<	3C	0011 1100
72	H	48	0100 1000	104	h	68	0110 1000	39	'	27	0010 0111	61	=	3D	0011 1101
73	I	49	0100 1001	105	i	69	0110 1001	40	(	28	0010 1000	62	>	3E	0011 1110
74	J	4A	0100 1010	106	j	6A	0110 1010	41	)	29	0010 1001	63	?	3F	0011 1111
75	K	4B	0100 1011	107	k	6B	0110 1011	42	*	2A	0010 1010	64	@	40	0100 0000
76	L	4C	0100 1100	108	l	6C	0110 1100	43	+	2B	0010 1011	91	[	5B	0101 1011
77	M	4D	0100 1101	109	m	6D	0110 1101	44	,	2C	0010 1100	92	\	5C	0101 1100
78	N	4E	0100 1110	110	n	6E	0110 1110	45	-	2D	0010 1101	93	]	5D	0101 1101
79	O	4F	0100 1111	111	o	6F	0110 1111	46	.	2E	0010 1110	94	^	5E	0101 1110
80	P	50	0101 0000	112	p	70	0111 0000	47	/	2F	0010 1111	95	_	5F	0101 1111
81	Q	51	0101 0001	113	q	71	0111 0001	48	0	30	0011 0000	96	`	60	0110 0000
82	R	52	0101 0010	114	r	72	0111 0010	49	1	31	0011 0001	123	{	7B	0111 1011
83	S	53	0101 0011	115	s	73	0111 0011	50	2	32	0011 0010	124		7C	0111 1100
84	T	54	0101 0100	116	t	74	0111 0100	51	3	33	0011 0011	125	}	7D	0111 1101
85	U	55	0101 0101	117	u	75	0111 0101	52	5	34	0011 0100	126	~	7E	0111 1110
86	V	56	0101 0110	118	v	76	0111 0110	53	5	35	0011 0101				
87	W	57	0101 0111	119	w	77	0111 0111								
88	X	58	0101 1000	120	x	78	0111 1000								
89	Y	59	0101 1001	121	y	79	0111 1001								
90	Z	5A	0101 1010	122	z	7A	0111 1010								

ตัวอย่างการเขียน ASCII ในภาษาซี(C)

```
#include <stdio.h>
```

```
int main() {
```

```
    int num;
```

```

printf("Enter a number (0-127): ");
scanf("%d", &num);
if (num >= 0 && num <= 127) {
    printf("The ASCII character for %d is: %c\n", num, num);
} else {
    printf("Please enter a number between 0 and 127.\n");
}
return 0;
}

```

อีกตัวอย่างนำมาประยุกต์ใช้กับการหาเลข 0 ตั้งแต่ 1 - 10000

```
#include <stdio.h>
```

```

int main() {
    int count = 0;
    char buffer[10];

    for (int i = 1; i <= 10000; i++) {
        sprintf(buffer, "%d", i); // แปลงตัวเลขเป็นสตริง
        for (int j = 0; buffer[j] != '\0'; j++) {
            if (buffer[j] == '0') { // เปรียบกับอักขระ '0' (ASCII 48)
                count++;
            }
        }
    }
    printf("จำนวนเลข 0 ที่พบทั้งหมด ตั้งแต่ 1 ถึง 10000 คือ: %d\n", count);
    return 0;
}

```

การใช้ GNU COMPLIER COLLECTION(GCC)

- เริ่มต้นจากการตรวจสอบว่า gcc ติดตั้งหรือยังโดยการใช้คำสั่ง -v หรือ -version คำสั่งเบื้องต้นที่ควรรู้เลยคือ
“gcc -o ชื่อไฟล์ใหม่ ชื่อไฟล์ที่ต้องการนำมาคอมไพล์”
- -o ย่อมาจาก output
- gcc เป็นคำสั่งหลักเพื่อเรียกใช้งานตัว compiler หรือ app compiler นั้นแหละ

ตัวอย่างการใช้งานจริง

1. gcc -o hello hello.c
2. gcc -o biggestOfThree biggestOfThree.c
3. ./hello เพื่อรันโปรแกรม
4. ./biggestOfThree เพื่อรันโปรแกรม